

W14D01 — SQLAlchemy ORM: Model Definition

Week 14 · Day 01 · 9 Jun 2026

1. Konsep Dasar — ORM sebagai Blueprint Digital

Bayangkan schema PostgreSQL kamu adalah blueprint gedung. Tabel adalah ruangan, kolom adalah dinding dan properti, foreign key adalah koridor penghubung antar ruangan.

SQLAlchemy ORM membaca blueprint itu, lalu membuat **representasi Python**-nya. Setiap tabel jadi Python class, setiap baris jadi instance, setiap kolom jadi attribute, dan setiap foreign key + JOIN jadi *relationship()* yang bisa dinagivasi dengan Python biasa tanpa menulis SQL manual.

Konsep SQL	Konsep ORM (Python)	Keterangan
TABLE predictions	class Prediction(Base):	Tabel → Class
id UUID PRIMARY KEY	id: Mapped[uuid.UUID]	Kolom → Attribute
Satu baris data	p = Prediction(...)	Row → Instance
FOREIGN KEY	relationship()	Link → Navigasi OOP

2. Setup SQLAlchemy 2.x

Install dependencies:

```
pip install sqlalchemy psycpg2-binary python-dotenv
```

database.py — Engine & Session

Engine = koneksi ke database. **Session** = unit of work: semua operasi DB dalam satu 'transaksi' Python.

echo=True sangat penting saat development — kamu bisa lihat SQL apa yang benar-benar dieksekusi ORM di balik layar.

```
from sqlalchemy import create_engine
from sqlalchemy.orm import DeclarativeBase, sessionmaker

engine = create_engine(
    os.getenv('DATABASE_URL'),
    echo=True,           # print semua SQL yang dieksekusi
    pool_size=5,         # max 5 koneksi aktif
    pool_pre_ping=True   # cek koneksi sebelum dipakai
)

class Base(DeclarativeBase):
    pass

SessionLocal = sessionmaker(
    autocommit=False,    # kamu yang kontrol kapan commit
    autoflush=False,     # jangan flush sebelum kamu minta
    bind=engine
)
```

DeclarativeBase (SQLAlchemy 2.x) lebih explicit dan type-safe dibanding declarative_base() versi lama. Semua model harus inherit dari class Base ini.

3. Model Definition — Mirror dari dss_schema.sql

Tiga model utama untuk Project 3 DSS, masing-masing mencerminkan tabel yang dibuat di W13D06:

AnalysisRequest — model utama (parent)

```
from sqlalchemy.dialects.postgresql import UUID, JSONB
from sqlalchemy.orm import Mapped, mapped_column, relationship

class AnalysisRequest(Base):
    __tablename__ = 'analysis_requests'

    id: Mapped[uuid.UUID] = mapped_column(
        UUID(as_uuid=True),
        primary_key=True,
        server_default=text('gen_random_uuid()') # PostgreSQL yang generate
    )
    title: Mapped[str] = mapped_column(String(255), nullable=False)
    parameters: Mapped[Optional[dict]] = mapped_column(JSONB, nullable=True)
    status: Mapped[str] = mapped_column(
        String(50), nullable=False, server_default='pending'
    )
    created_at: Mapped[datetime] = mapped_column(
        DateTime(timezone=True), server_default=func.now()
    )
    updated_at: Mapped[datetime] = mapped_column(
        DateTime(timezone=True), server_default=func.now(), onupdate=func.now()
    )
    # Relationship ke child – cascade mirror ON DELETE CASCADE SQL
    scenarios: Mapped[list['ScenarioResult']] = relationship(
        'ScenarioResult',
        back_populates='request',
        cascade='all, delete-orphan'
    )
```

ScenarioResult — child model

```

class ScenarioResult(Base):
    __tablename__ = 'scenario_results'

    id: Mapped[uuid.UUID] = mapped_column(
        UUID(as_uuid=True), primary_key=True,
        server_default=text('gen_random_uuid()')
    )
    request_id: Mapped[uuid.UUID] = mapped_column(
        UUID(as_uuid=True),
        ForeignKey('analysis_requests.id', ondelete='CASCADE'), # string, bukan class name
        nullable=False
    )
    scenario_type: Mapped[str] = mapped_column(String(50), nullable=False)
    risk_score: Mapped[Optional[Decimal]] = mapped_column(
        Numeric(precision=5, scale=4), nullable=True # Decimal bukan float!
    )
    output_data: Mapped[Optional[dict]] = mapped_column(JSONB, nullable=True)
    request: Mapped['AnalysisRequest'] = relationship(
        'AnalysisRequest', back_populates='scenarios'
    )

```

4. Konsep Kritis yang Wajib Dipahami

server_default vs default (Python)

	server_default	default (Python)
Diset oleh	PostgreSQL	Python/SQLAlchemy
Kapan berjalan	Saat SQL INSERT dieksekusi	Saat objek Python dibuat
Konsisten dari psql CLI	✓ Ya	✗ Tidak
Contoh	func.now(), gen_random_uuid()	default=datetime.now

cascade='all, delete-orphan' vs ForeignKey ondelete='CASCADE'

Keduanya 'hapus child kalau parent dihapus' — tapi siapa yang mengeksekusi berbeda:

	cascade (ORM)	ondelete CASCADE (SQL)
Dieksekusi oleh	Python / SQLAlchemy	PostgreSQL langsung
Berlaku saat	Hapus via session Python	Hapus dari mana saja (psql, tool lain)
Butuh ORM?	Ya	Tidak
Rekomendasi	Selalu pasang keduanya	Selalu pasang keduanya

Mengapa relationship() tidak ada di SQL?

PostgreSQL tidak peduli dengan OOP. Jika ingin data gabungan, developer cukup menulis INNER JOIN. Di Python, kita tidak ingin menulis JOIN manual setiap saat — kita ingin memperlakukan data seperti objek Python biasa.

relationship() adalah fitur kenyamanan murni milik Python. Database fisik di bawahnya hanya peduli pada ForeignKey — **relationship()** hanya hidup di layer ORM dan tidak menghasilkan DDL apapun di PostgreSQL.

5. Gotcha Penting

UUID terisi None sebelum flush/commit

Karena id menggunakan server_default (`gen_random_uuid()` di PostgreSQL), Python tidak tahu nilainya sampai data benar-benar dikirim ke database:

```
req = AnalysisRequest(title='Test')
session.add(req)
print(req.id)    # → None    (belum ke DB)

session.flush() # kirim ke DB dalam transaksi, tapi belum commit
print(req.id)    # → UUID terisi    (PostgreSQL sudah eksekusi gen_random_uuid())

session.commit() # 'kunci' transaksi permanen
```

`flush()` berguna saat kamu butuh ID sebelum transaksi selesai, misalnya saat insert parent dan langsung butuh ID-nya untuk insert child.

Mapped[float] tidak cocok dengan Numeric

Jangan campur tipe Python dengan tipe kolom yang tidak sesuai:

```
# ■ SALAH – float tidak presisi eksak
accuracy: Mapped[float] = mapped_column(Numeric(5, 4))

# ✓ BENAR – Decimal = presisi eksak, cocok dengan Numeric
from decimal import Decimal
accuracy: Mapped[Optional[Decimal]] = mapped_column(Numeric(5, 4), nullable=True)
```

JSONB harus dari dialect PostgreSQL

```
# ■ SALAH – JSON generic tidak bisa di-index
from sqlalchemy import JSON

# ✓ BENAR – JSONB PostgreSQL: bisa di-index, bisa query per field
from sqlalchemy.dialects.postgresql import JSONB
```

6. Verifikasi — create_all

Untuk testing lokal, `Base.metadata.create_all(engine)` membuat semua tabel yang belum ada. Ini tidak mengganggu tabel yang sudah ada — hanya membuat yang baru. **Bukan untuk production** — production menggunakan Alembic migration (W14D02).

```
# init_db.py
from database import engine, Base
import models # noqa: F401 — agar Base.metadata 'tahu' semua model

Base.metadata.create_all(bind=engine)
# Verifikasi: docker exec -it pg-dss psql -U dss_user -d dss_db
# Lalu: \d analysis_requests
```

7. Quick Reference — Tipe Data

Kebutuhan	Tipe SQLAlchemy	Mapped[...] Python	Catatan
Primary key	<code>UUID(as_uuid=True)</code>	<code>uuid.UUID</code>	Import dari <code>dialects.postgresql</code>
Teks pendek	<code>String(n)</code>	<code>str</code>	n = max karakter
Angka finansial/presisi	<code>Numeric(p, s)</code>	<code>Decimal</code>	Bukan float!
Angka ML score	<code>Float</code>	<code>float</code>	Efisiensi komputasi
JSON semi-structured	<code>JSONB</code>	<code>dict</code>	Dialect PostgreSQL
Boolean	<code>Boolean</code>	<code>bool</code>	<code>server_default=text('FALSE')</code>
Timestamp	<code>DateTime(timezone=True)</code>	<code>datetime</code>	Selalu <code>timezone=True</code>